VentureGraph — Methodology Appendix

VentureGraph 2.0 · The FinTech Foundry Edition

C-DE422 · Big Data Engineering II · Egypt University of Informatics
Mohamed Hares · May 2026

Methodology Appendix — VentureGraph 2.0

Author: Mohamed Hares · **Course:** C-DE422 · **Date:** May 2026 **Companion to:** `REPORT.md` · **Pre-registration:** `preregistration.py` (SHA-256 locked)

A.1 The Four-Layer Stack

VentureGraph's empirical pipeline is four layers stacked on a single audit backbone.

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
LAYER 4: BRIDGE → FF5 abnormal sector alpha (α_{s, t+k}) ■ ■
panel_regression_final.py ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
LAYER 3: SIGNAL → SMS(s, t) · SSI(s, t) ■ ■ sms_engine.py · compute_ssi.py ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
LAYER 2: SCORE → TPS(investor, t) (point-in-time) ■ ■ tps.py ·
expanding_window_tps.py ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
LAYER 1: GRAPH → G_t = (V_t, E_t, w_t) directed, decayed ■ ■ graph_construction.py
```

```
■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
AUDIT BACKBONE → SHA-256( protocol, inputs, eval_date ) ■ ■ preregistration.py +
expanding_window_tps.py:79-95 ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Every numeric output above the backbone is reproducibly computable from the inputs below it. The backbone emits a SHA-256 hash for every evaluation, providing a non-repudiable audit trail.

A.2 Layer 1 — The Co-Investment Graph

File: `graph_construction.py`

Definition. A directed, time-weighted graph $G = (V, E)$ where:

- V = investors who participated in Series A or B rounds in `{finance, fintech, software, saas, enterprise}` on any company in the Crunchbase 2013 snapshot.
- E = ordered pairs $(A \to B)$ such that on some company c, investor A entered a round that preceded investor B's round.
- $w(A \to B, c) = e^{-\lambda \Delta t}$ with $\lambda = 0.3$ and Δt measured in years.

Scale. 1,101 nodes, 3,870 edges after pruning. Density 0.0032. Mean clustering 0.5147. Diameter (largest WCC) 14.

Key design choice. The direction $A \to B$ encodes *temporal precedence*, not causality or endorsement. A got there first; B followed. The decay weight $e^{-0.3 \Delta t}$ reflects the decaying informational content of a long-ago precedence.

A.3 Layer 2 — The Temporal Precursor Score (TPS)

Files: `tps.py`, `expanding_window_tps.py`

Static definition. For each investor A:

$$ \text{TPS}(A) = \frac{1}{|\text{portfolio}(A)|} \sum_{c \in \text{portfolio}(A)} \sum_{B \in \text{top-tier}} w(A \to B, c) \cdot \text{rank}^{-1}(B) $$

where: - top-tier = investors in the top 20% of global in-weighted-degree (pre-registered as p80); - $\text{rank}^{-1}(B)$ = inverse positional rank of B in their own entry-date ordering on company c (earlier follower = higher weight).

Interpretation. TPS is *volume-independent*. An investor with one deal where they precede Sequoia has a higher TPS than an investor with ten deals where they follow everyone. TPS measures prescience, not prominence. The empirical rank correlation with out-degree is $\rho \approx 0.31$ — demonstrating that TPS captures information distinct from deal count.

Point-in-time extension (`expanding_window_tps.py`). To eliminate look-ahead bias, TPS is recomputed at each quarterly evaluation date t using **only** edges with $\text{round_date} \le t$. This produces a panel `tps_panel_expanding.csv` with one (investor, eval_date, TPS) row per observation. Each row carries a SHA-256 hash of its input graph state — the contamination firewall.

A.4 Layer 3 — The Sector Signals

A.4.1 Swarm Signal Intensity (SSI)

File: `compute_ssi.py`

$$ \text{SSI}(s, t) = \sum_{i \in \text{new-entrants}(s, t-w : t)} \text{TPS}_t(i) \cdot \text{IPD-weight}(i, s, t) $$

Captures: *how much prescient capital entered sector s in the rolling w-month window ending at t?* The pre-registered window is $w = 6$ months; IPD-weighting uses rank-inverse by entry date within the window.

A.4.2 Smart Money Silence (SMS) — the bonus innovation

File: `sms_engine.py`

The primary conceptual contribution of the project.

Algorithm. 1. At eval date t, identify the set of top-tier investors $\mathcal{T}_t$ using `TPS_t` (top 30% by TPS at that date). 2. For every company c in sector s that raised a Series A in $[t - 12 \text{mo}, t]$, compute the *expected* set of top-tier follow-ons in a Series B: $$\text{Expected}_c = { i \in \mathcal{T}_t : i \text{ invested in } c \text{ at Series A} }$$ 3. For every company c that raised a Series B in $[t - 3 \text{mo}, t]$, compute the *realized* top-tier follow-on set. 4. **Silence** = $\text{Expected}_c \setminus \text{Realized}_c$ — top-tier investors who led Series A but *did not* follow to Series B. 5. Aggregate over the sector: $$\text{SMS}(s, t) = \frac{|\text{Total silences in } s \text{ at } t|}{|\text{Total expected top-tier follow-ons}|}$$

Interpretation. SMS quantifies *structural abstention* from elite capital — the inverse of link formation. A high SMS in sector s at t says: *"the smart money that was here is no longer showing up."* This is the paper's proposed bearish signal.

A.5 Layer 4 — The Public-Equity Bridge

File: `panel_regression_final.py`, `ff5_regression.py`

Step 1 — abnormal returns. For each sector ETF $s \in {\text{XLF, IGV, FDN, XBI, SOXX}}$, estimate a rolling 24-month Fama-French 5-factor model and extract the monthly abnormal return (alpha) residual.

Step 2 — horizon alignment. For each sector-quarter observation, forward-align the alpha residual by $k \in {6, 12, 18, 24, 36}$ months.

Step 3 — panel regression. $$ \alpha_{s, t+k} = \beta_0 + \beta_1 \cdot \text{SSI}*{s, t} + \beta_2 \cdot \text{SSI-dumb}*{s, t} + \gamma_s + \delta_t + \epsilon_{s, t} $$

with sector and time-quarter fixed effects and two-way clustered standard errors (sector × time, Cameron–Gelbach–Miller).

Step 4 — robustness battery (pre-registered in `preregistration.py:170-223`): - ROB-01: GFC exclusion (drop 2008–2009) - ROB-02: placebo date shuffle (1,000 permutations) - ROB-03: alternative SSI window (3, 9 months) - ROB-04: secondary ETF mapping - ROB-05: investor-type subsamples - ROB-06: TPS percentile sensitivity (p70, p90)

Wild cluster bootstrap (Rademacher, n=999) produces a bootstrapped standard error and p-value that are robust to few-cluster asymptotics.

A.6 The Audit Backbone

File: `expanding_window_tps.py:79-95` · **Function:** `_compute_hash(...)`

Every point-in-time TPS computation emits:

```
{ "eval_date": "YYYY-MM-DD", "input_hash": sha256(graph_edges_until_eval_date),
"protocol_hash": sha256(preregistration.py), "output_hash": sha256(tps_values),
"timestamp": utc_now() }
```

This tuple is what a sovereign-grade compliance officer requires to trust a signal. *The score was this; on that date; computed from that data; under that protocol. Proof: three matching hashes.*

Why it matters. Pitchbook, CB Insights, and similar platforms publish numbers without this chain. A regulator asking *"on what data was this score computed, and has the protocol since been modified?"* cannot be answered by those platforms. It can be answered here by running `verify_protocol_integrity(hash)` at the top of any analysis script.

A.7 Reproducibility Checklist

Step	Command	Expected artifact
1	`python graph_construction.py`	`edges.csv`, graph PNG
2	`python tps.py`	`tps_scores.csv`
3	`python expanding_window_tps.py`	`tps_panel_expanding.csv` (per-snapshot SHA-256)

4	`python sna_metrics.py`	console: density, diameter, clustering
5	`python centrality_comparison.py`	`centrality_comparison.csv` + PNG
6	`python community_detection.py`	`community_partition.csv`, `community_summary.csv`, PNG
7	`python link_prediction.py`	`link_predictions.csv`, 3 PNGs
8	`python sms_engine.py`	`sms_scores.csv`, `sms_silence_events.csv`, `sms_alpha_correlation.csv`
9	`python compute_ssi.py`	`ssi_events.csv`, `event_panel.csv`
10	`python panel_regression_final.py`	console: regression tables, robustness battery
11	`python preregistration.py`	`protocol_seal.json` with locked hash
12	`streamlit run app.py`	interactive dashboard (Academic + Sovereign modes)

All steps are idempotent: re-running produces identical outputs given identical inputs. Any divergence indicates the protocol has been modified (caught by `verify_protocol_integrity`).

A.8 Mapping to C-DE422 Deliverables

Brief section	Method layer	Files
Part A — Network statistics	Layer 1	`graph_construction.py`, `sna_metrics.py`
Part B — Centrality	Layer 2	`tps.py`, `centrality_comparison.py`
Part C — Community detection	Layer 1 (derived)	`community_detection.py`
Part D — Advanced analysis	Layer 3	`link_prediction.py`, `sms_engine.py`

Part E — Visualizations	Derived	9 PNGs in repo
Part F — Interactive dashboard	All layers	`app.py` (Academic + Sovereign modes)
Written report	Synthesis	`REPORT.md` + this appendix
5-mark bonus	Layer 3+4	`sms_engine.py`, `panel_regression_final.py`, `preregistration.py`

End of Methodology Appendix.